



Diploma in Software Engineering 26.1F
Module: DATA MANAGEMENT 1

Final Report

B2B TEXTILE INVENTORY AND SALES MANAGEMENT SYSTEM
Organization: Colour Selection – Textile Trading and Reselling

Name	Registration No.
S.R.R MOULANA	CODSE261F-036
A.M.A ZIMTHI	CODSE261F-033
S.W EKANAYAKE	CODSE261F-040
K.P.H DILRUWAN	CODSE261F-055

Lecturer: Mr. Shashika Bandara
Submission Date :03rd October 2026

Abstract

This project is about creating a database for a company called Colour Selection. The company is in the business of buying and selling textiles to businesses. The main goal is to replace spreadsheet records with a single clean database. The main goal is to make things easier, for the company. This database will help the company manage suppliers. This database will help the company manage inventory in warehouses. This database will help the company manage sales orders from customers. This database will help the company manage bills and payments.

The project started by figuring out the rules the company follows. Then nine important parts of the business were identified. These parts and how they connect were shown in a diagram called an ER diagram. This diagram used a style called Chens notation to show how each part relates to others. All the rules about how times each part can connect were made clear in the diagram.

After that the diagram was turned into a database structure. This structure was made to follow the Third Normal Form so that there is no repeated data.. Sometimes some data was kept in a less organized way to make sure that financial numbers are accurate over time.

The database was built in Microsoft SQL Server. To make sure the data is correct the database used rules, like primary keys foreign keys and checks. Then some fake data was added to the database to test it. A lot of kinds of queries were written so that the managers can get the information they need. These queries ranged from ones to complex ones that bring data from many parts of the database together.

Security was also important. Different roles were created. The manager role had access to add, change, delete and see all data.. The clerk role had limited access. Clerks could not delete data or change basic information that the database relies on.

In the end the database helps Colour Selection manage everything efficiently. It keeps the supply chain and billing processes accurate. It also helps the company run smoothly every day and gives managers the reports they need.

Acknowledgement

We would like to express our gratitude to everyone who supported us in completing this Data Management 1 coursework, the design and implementation of the B2B Textile Inventory and Sales Management System for Colour Selection.

First we thank our lecturer, Mr. Shashika Bandara for his guidance the coursework brief and the constructive feedback that shaped every stage of this project. His lessons on the model normalization and SQL were the foundation of our work. He helped us select the organization identify the entities and build and secure the database.

We also thank the administrative staff of the National Institute of Business Management (NIBM) for providing the learning environment the laboratory facilities and the Microsoft SQL Server tools used throughout the project. Their support made it possible to work on the system effectively and efficiently.

Our appreciation goes to every member of the group. The work was divided clearly with each member responsible for a part of the database design and implementation the project report, the project proposal and the presentation. Each contribution was essential.

Finally we thank our group members for their patience and encouragement while we completed this coursework. Their support kept us going during the **times**.

Table of Contents

1.Abstract.....	2
2.Acknowledgemen	3
3.Table of Contents	4
4.List of Tables	5
5.List of Figures.....	6
6.Chapter 1 – Introduction.....	7
1.1 Background	7
1.2 Organization Profile	7
1.3 Problem Statement.....	7
1.4 Aim and Objectives	8
1.5 Tools and Technologies.....	8
7.Chapter 2 – Analysis.....	9
2.1 Requirements Gathering Approach	9
2.2 Functional Requirements.....	9
2.3 Non functionals and Security Requirements	9
2.4 Business Rules.....	10
2.5 Entity Identification.....	10
2.6 Information Requirements.....	11
2.7 Assumptions	11
8.Chapter 3 – Design and Implementation	12
3.1 Conceptual Design.....	12
3.2 Relationships and Cardinality.....	13
3.3 Participation Constraints.....	13
3.4 Relational Schema Mapping.....	14
3.4.1 Final Relational Schema	15
3.5 Data Dictionary	16
3.6 Normalization.....	17
3.7 Physical Database Design (DDL)	19
3.8 Data Manipulation Language (DML).....	21
3.9 Data Modification (UPDATE)	22
3.10 SQL Queries	22
3.10.1 Basic SELECT Queries	22
3.10.2 Advanced SELECT Queries	24
3.10.3 JOIN Queries	26
3.10.4 Database Security (DCL).....	27
9.Chapter 4 – Conclusion	29
4.1 Summary	29
4.2 Achievement of Objectives	29
4.3 Final Remarks.....	29
10.Contribution Table	30

List of Figures

Figure No.	Page
Figure 1 ER Diagram	12
Figure 2 Finalized Relational Schema.....	15
Figure 3.1: Entity Relationship (ER) diagram of the Colour Selection database	12
Figure 3.2: ColourSelectionDB and its tables in SQL Server Management Studio.....	19
Figure 3.3: Record count of every table.....	20
Figure 3.4: Output of Query B1.....	21
Figure 3.5: Output of Query B2.....	22
Figure 3.6: Output of Query B3.....	22
Figure 3.7: Output of Query B4.....	22
Figure 3.8: Output of Query B5.....	23
Figure 3.9: Output of Query A1	23
Figure 3.10: Output of Query A2	23
Figure 3.11: Output of Query A3	24
Figure 3.12: Output of Query A4	24
Figure 3.13: Output of Query A5	24
Figure 3.14: Output of Query J1	25
Figure 3.15: Output of Query J2	25
Figure 3.16: Output of Query J3	26
Figure 3.17: Database users and permissions in SSMS.....	27

List of Tables

Table No.	Page
<i>Table 1 Customer.....</i>	<i>14</i>
<i>Table 2 Supplier.....</i>	<i>14</i>
<i>Table 3 Warehouse.....</i>	<i>14</i>
<i>Table 4 Product.....</i>	<i>14</i>
<i>Table 5 Inventory.....</i>	<i>14</i>
<i>Table 6 Sales_Order.....</i>	<i>14</i>
<i>Table 7 Invoice.....</i>	<i>14</i>
<i>Table 8 Order_Detail.....</i>	<i>14</i>
<i>Table 9 Payment.....</i>	<i>14</i>

Chapter 1: Introduction

1.1 Background

I can see how the textile industry moves amounts of raw materials such as cotton, silk, linen, synthetic blends and industrial thread from suppliers through warehouses to garment manufacturers and fashion retailers. A trading business, in this chain must always know what the trading business has what the trading business has in stock, where the trading business keeps the stock what customers have ordered and which invoices are still unpaid. When these facts are kept in spreadsheets and paper ledgers they quickly become inconsistent, duplicated and hard to find. A Relational Database Management System (RDBMS) solves this problem by storing each fact linking related facts through keys and applying rules that keep the data correct. This project uses those ideas to model a business case.

1.2 Organization Profile

Colour Selection is a business-to-business textile trading and reselling company. It buys quantities of raw materials and fabrics from suppliers both in the local market and abroad. The company keeps these goods stored in regional warehouses. Then it sells the fabrics to garment manufacturers, boutiques and clothing brands. The company runs two workflows every day and they are connected.

- **Supply chain flow:** In this flow suppliers send products to Colour Selection. The products are then stored as inventory, in one or more of the company's warehouses. This ensures that materials are available when needed.
- **Sales and billing flow:** Here customers place sales orders. Each order has order lines that list the items being bought. After the order is confirmed an invoice is issued to the customer. Payments are then. Settled. This flow connects directly to the supply chain because the inventory is updated when sales occur.

1.3 Problem Statement

Colour Selection currently tracks its work using scattered spreadsheets and old-style paper records. This creates three issues:

- **Inventory blind spots:** it is hard to see how much stock is in each warehouse all, in one place, which causes selling items that are not available or running out of products.
- **Disconnected billing:** orders, bills and money are kept in places so it is difficult to find which bills have not been paid and which orders are still waiting.
- **Supplier mismanagement:** there is no place that shows which product goes with which supplier, which makes it slow to place new orders.

1.4 Aim and Objectives

Aim: to design and implement a database in Microsoft SQL Server that supports the main trading process of Colour Selection. The specific objectives are:

- To find least eight different entities and create an Entity Relationship (ER) diagram that shows how they connect and how many relationships exist between them.
- To turn the ER diagram into a structure that uses keys and foreign keys and is in Third Normal Form.
- To build the database and tables in SQL Server with the needed rules and restrictions.
- To fill every table with least ten entries of real-life data.
- To write SQL queries that include commands, complex ones with GROUP BY HAVING ORDER BY and joins between multiple tables to answer common business questions.
- To protect the data by setting up users with levels of access and permissions.

1.5 Tools and Technologies

Table 1.1: Tools and technologies used

Tool	Purpose
Microsoft SQL Server	Relational database engine that hosts ColourSelectionDB.
SQL Server Management Studio (SSMS)	Used to run the DDL, DML and DCL scripts and to capture query output screenshots.
Transact-SQL (T-SQL)	SQL dialect used for table creation, data population, queries and security.
ER diagram (Bristol board and digital copy)	Conceptual design drawn with entities, relationship diamonds, cardinality ratios and participation constraints.
Microsoft Word and PowerPoint	Preparation of the project report and the final presentation.

Chapter 2: Analysis

2.1 Requirements Gathering Approach

The needs came from the business situation of choosing colors and from the assignment given in class. The group looked at how a fabric seller buys keeps, sells and charges for items wrote down the questions managers ask each day. Changed each question into a data need. This led to the needs the rules for the business and the list of things, in this chapter. Every need was then connected to a table and a query to make sure nothing was missed.

2.2 Functional Requirements

Table 2.1: Functional requirements

ID	Requirement	Description
FR1	Customer management	Store customer name, phone, email and address; add and update customers.
FR2	Supplier management	Store supplier contact details and link each product to its supplier.
FR3	Product catalogue	Maintain product name, type, unit price, color and size.
FR4	Warehouse and stock tracking	Record warehouse capacity and the quantity of each product held in each warehouse.
FR5	Sales order processing	Record orders with date, status and total, and the products and quantities on each order.
FR6	Invoicing	Raise invoices for customers and track whether each is paid or unpaid.
FR7	Payment recording	Record payments with date, amount, method and status against an order and invoice.
FR8	Reporting	Answer business questions on sales, stock, pricing and outstanding payments.
FR9	Access control	Restrict what each type of user can view, change or delete.

2.3 Non-Functional and Security Requirements

Sales and financial data is very important for the business. Access must be limited to only what is needed. Two roles are needed: a Manager who has all the data access and a Clerk who handles transactions but cannot delete orders, invoices, payments or customers. The Clerk also cannot change master data, like products, suppliers and warehouses. The other quality requirements are listed below.

Table 2.2: Non-functional requirements

ID		Description
NFR1	Security	Access is limited by role (Manager and Clerk) using GRANT and DENY permissions.
NFR2	Data integrity	Primary keys, foreign keys, NOT NULL and CHECK constraints prevent invalid or orphan records.
NFR3	Consistency	Every fact is stored once; the schema is normalized to Third Normal Form.

NFR4	Accuracy	Money values use DECIMAL, and the selling price is copied into each order line to preserve history.
NFR5	Performance	Primary keys are indexed automatically, and stored totals speed up reporting queries.
NFR6	Maintainability	Naming standards are applied (upper-case table names, PascalCase column names, ID suffix for keys) and the script is fully commented.
NFR7	Scalability	New customers, products, suppliers and warehouses can be added without changing the structure.

2.4 Business Rules

Table 2.3: Business rules of the system

No.	Business rule
BR1	A supplier can supply many products, but each product is supplied by exactly one supplier.
BR2	a warehouse that can hold inventory records. A product can appear in the inventory of warehouses. Each inventory record belongs to one warehouse and one product.
BR3	A customer can place many sales orders, but each sales order belongs to exactly one customer.
BR4	A sales order contains one or more order details; each order detail belongs to exactly one order and refers to exactly one product.
BR5	A customer can receive many invoices, but each invoice is issued to exactly one customer.
BR6	A payment is recorded against exactly one sales order and one invoice; an order or invoice may have many payments.
BR7	Order status is Pending, Processing or Completed; invoice status is Paid or Unpaid.
BR8	The unit price is copied into each order detail, so later price changes do not alter past sales. SubTotal equals Quantity multiplied by UnitPrice.
BR9	Only the manager may delete records. Clerks may view and process transactions but are denied deletion of business records.

2.5 Entity Identification

Nine entities were identified. This meets the requirement of eight. Each one is a thing that the business needs to keep data about. Foreign keys are not listed as attributes here. They are added later when the schema is being mapped.

Table 2.4: Identified entities and attributes

Entity	Description	Attributes
CUSTOMER	Business that buys textiles	CustomerID, CustomerName, Phone, Email, Address
SUPPLIER	Business that provides products	SupplierID, SupplierName, Phone, Email, Address
WAREHOUSE	Storage location for stock	WarehouseID, WarehouseName, Location, Capacity
PRODUCT	Fabric, thread or raw material sold	ProductID, ProductName, ProductType, UnitPrice, Color, Size
INVENTORY	Stock of a product in a warehouse	InventoryID, QuantityAvailable, LastUpdated
SALES_ORDER	Order placed by a customer	OrderID, OrderDate, OrderStatus, TotalAmount (derived)
ORDER_DETAIL	One product line of an order	OrderDetailID, Quantity, UnitPrice, SubTotal (derived)
INVOICE	Bill issued to a customer	InvoiceID, InvoiceDate, TotalAmount (derived), Status
PAYMENT	Payment made against an order and invoice	PaymentID, PaymentDate, Amount, PaymentMethod, PaymentStatus

2.6 Information Requirements

Management needs quick answers to frequently asked questions. These were grouped into three categories and implemented as SQL queries in Section 3.10.

Table 2.5: Information requirements and query categories

Category	Queries	Typical questions answered
Basic SELECT	5	Customer contact list, fabric products, pending orders, warehouse capacity, unpaid invoices.
Advanced SELECT (GROUP BY, HAVING, ORDER BY)	5	High-value customers, average price by product type, stock per warehouse, units sold per product, invoices by status.
JOIN queries	3	Products with warehouse stock, customers with their ordered items, invoices with customer and payment details.

2.7 Assumptions

A1. Every product is supplied by one supplier. A supplier may be registered before a supplier supplies any product (BR1).

A2. INVENTORY is an entity that holds one record for each product in each warehouse resolving the many-to-many link between WAREHOUSE and PRODUCT (BR2).

A3. A sales order has one or more order details each tied to one product. ORDER_DETAIL resolves the to-many link between SALES_ORDER and PRODUCT (BR4).

A4. A customer may place orders and receive many invoices; every order and every invoice belongs to exactly one customer (BR3, BR5).

A5. A payment is recorded against one order and exactly one invoice. An order or invoice may have no payment yet or many payments (BR6).

A6. UnitPrice is copied into ORDER_DETAIL so that later price changes do not alter past sales (BR8).

A7. SubTotal and TotalAmount are derived values ($\text{Quantity} \times \text{UnitPrice}$ and the sum of the lines). They are stored deliberately for reporting. To preserve the billed figure.

A8. Order status is Pending, Processing or Completed; invoice status is Paid or Unpaid (BR7).

A9. Human Resources and employees are out of scope so orders are not assigned to a sales representative.

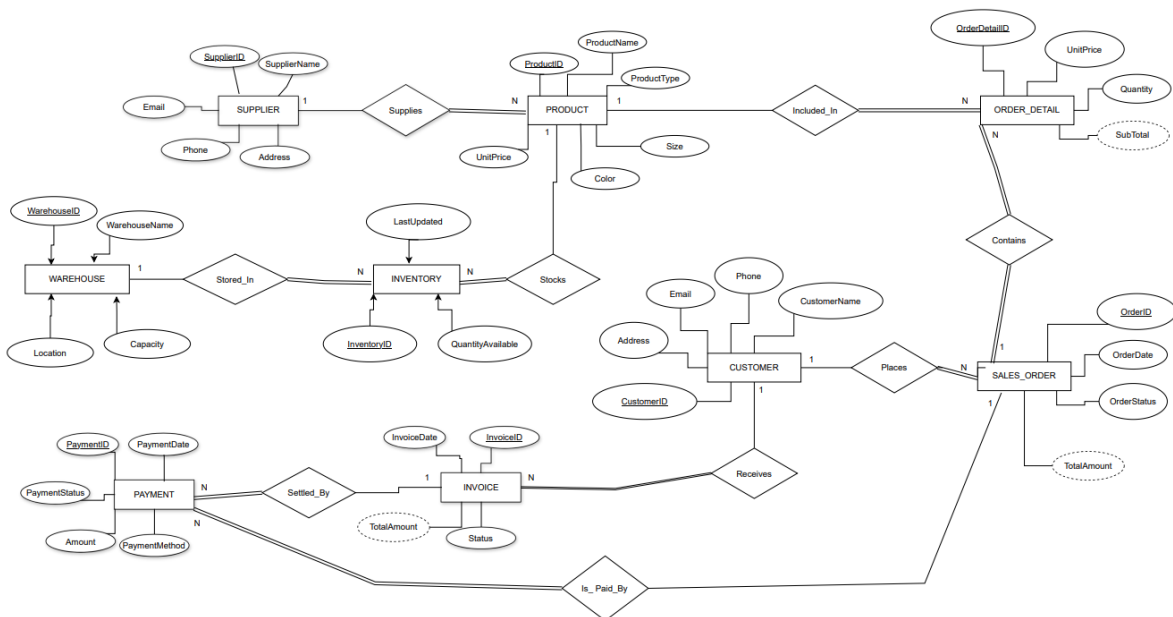
Chapter 3: Design and Implementation

The design goes through three steps of database design. The first step is design. This is, about what data's there and how it connects. The second step is design. This is how the data is put into tables. The third step is design. This is how the data is created and protected in SQL Server.

3.1 Conceptual Design

I have looked at the model that uses the nine entities of Table 2.4 and I see that the diagram is drawn in Chen notation. In this diagram entities are shown as rectangles relationships are shown as diamonds and attributes are shown as ovals. The conceptual model marks primary keys in the diagram with underlining and the model draws derived attributes such as SubTotal and TotalAmount in the diagram with dashed ovals. The conceptual model writes cardinality ratios, as 1 and N beside each relationship. The conceptual model marks total participation with a double line. The conceptual model uses INVENTORY. As associative entities to resolve the many-to-many relationships between WAREHOUSE and PRODUCT and, between SALES_ORDER and PRODUCT. The conceptual model shows the diagram with assumptions in Figure 3.1.

Figure 3.1: Entity Relationship (ER) diagram of the Colour Selection database



3.2 Relationships and Cardinality

The relationships and cardinality ratios come from the business rules. All nine relationships are one-, to-many or 1:N. That means each foreign key goes on the side of the relationship.

Table 3.1: Relationships and cardinality ratios

Relationship	Entity A	Entity B	Cardinality	Foreign key location
Supplies	SUPPLIER	PRODUCT	1 : N	PRODUCT.SupplierID
Holds	WAREHOUSE	INVENTORY	1 : N	INVENTORY.WarehouseID
Is stocked as	PRODUCT	INVENTORY	1 : N	INVENTORY.ProductID
Places	CUSTOMER	SALES_ORDER	1 : N	SALES_ORDER.CustomerID
Contains	SALES_ORDER	ORDER_DETAIL	1 : N	ORDER_DETAIL.OrderID
Is sold in	PRODUCT	ORDER_DETAIL	1 : N	ORDER_DETAIL.ProductID
Is billed by	CUSTOMER	INVOICE	1 : N	INVOICE.CustomerID
Is paid by	SALES_ORDER	PAYMENT	1 : N	PAYMENT.OrderID
Is settled by	INVOICE	PAYMENT	1 : N	PAYMENT.InvoiceID

3.3 Participation constraints

Participation is total when every entity must take part in the relationship, which is shown by a line. Participation is partial when an entity may not take part, which is shown by a line.

Table 3.2: Participation constraints

Relationship	Entity A	Entity B	Reason
Supplies	SUPPLIER: Partial	PRODUCT: Total	A supplier may be registered before supplying anything; every product has exactly one supplier.
Holds	WAREHOUSE: Partial	INVENTORY: Total	A new warehouse may be empty; every inventory record belongs to a warehouse.
Is stocked as	PRODUCT: Partial	INVENTORY: Total	A new product may not be stored yet; every inventory record is for a product.
Places	CUSTOMER: Partial	SALES_ORDER: Total	A customer may not have ordered yet; every order has a customer.
Contains	SALES_ORDER: Total	ORDER_DETAIL: Total	An order has at least one line; every line belongs to an order.
Is sold in	PRODUCT: Partial	ORDER_DETAIL: Total	A product may never have been sold; every line refers to a product.
Is billed by	CUSTOMER: Partial	INVOICE: Total	A customer may have no invoice yet; every invoice has a customer.
Is paid by	SALES_ORDER: Partial	PAYMENT: Total	An order may not be paid yet; every payment is for an order.
Is settled by	INVOICE: Partial	PAYMENT: Total	An invoice may not be paid yet; every payment settles an invoice.

3.4 Relational Schema Mapping

CUSTOMER

<u>CustomerID</u>	CustomerName	Phone	Email	Address
-------------------	--------------	-------	-------	---------

Table 1: Customer

SUPPLIER

<u>SupplierID</u>	SuupplierName	Phone	Email	Address
-------------------	---------------	-------	-------	---------

Table 2: Supplier

WAREHOUSE

<u>WarehouseID</u>	WarehouseName	Location	Capacity
--------------------	---------------	----------	----------

Table 3: Warehouse

PRODUCT

<u>ProductID</u>	ProductName	ProductType	UnitPrice	Color	Size	<u>SupplierID</u>
------------------	-------------	-------------	-----------	-------	------	-------------------

Table 4: Product

INVENTORY

<u>InventoryID</u>	QuantityAvailable	LastUpdated	<u>ProductID</u>	<u>WarehouseID</u>
--------------------	-------------------	-------------	------------------	--------------------

Table 5: Inventory

SALES_ORDER

<u>OrderID</u>	OrderDate	OrderStatus	TotalAmount	<u>CustomerID</u>
----------------	-----------	-------------	-------------	-------------------

Table 6: Sales Order

INVOICE

<u>InvoiceID</u>	InvoiceDate	TotalAmount	<u>Status</u>	<u>CustomerID</u>
------------------	-------------	-------------	---------------	-------------------

Table 7: Invoice

ORDER_DETAIL

<u>OrderDetailID</u>	Quantity	UnitPrice	SubTotal	<u>OrderID</u>	<u>ProductID</u>
----------------------	----------	-----------	----------	----------------	------------------

Table 8: Order Detail

PAYMENT

<u>PaymentID</u>	PaymentDate	Amount	PaymentMethod	PaymentStatus	<u>OrderID</u>	<u>InvoiceID</u>
------------------	-------------	--------	---------------	---------------	----------------	------------------

Table 9: Payment

3.4.1 Final relational Schema

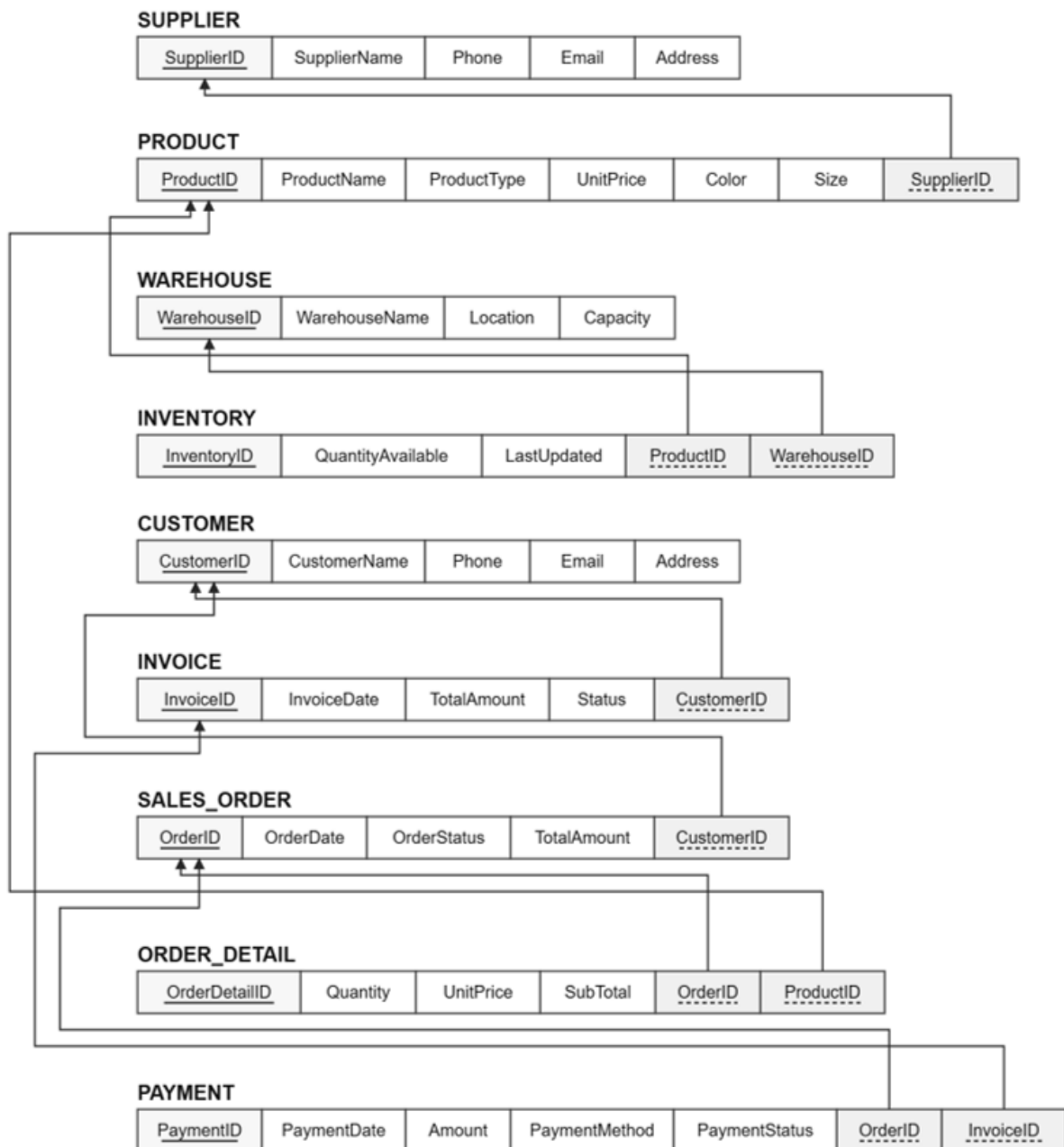


Figure 2: Final Relational Schema

3.5 Data Dictionary

Table 3.5 lists every column with its data type, constraint and meaning.

Column	Data Type	Constraint	Description
CUSTOMER – Business customers that buy textiles			
CustomerID	INT	PK, NOT NULL	Unique customer identifier
CustomerName	VARCHAR(100)	NOT NULL	Name of the customer business
Phone	VARCHAR(20)	–	Contact phone number
Email	VARCHAR(100)	–	Contact email address
Address	VARCHAR(200)	–	Business address
SUPPLIER – Businesses that supply products			
SupplierID	INT	PK, NOT NULL	Unique supplier identifier
SupplierName	VARCHAR(100)	NOT NULL	Name of the supplier
Phone	VARCHAR(20)	–	Supplier phone number
Email	VARCHAR(100)	–	Supplier email address
Address	VARCHAR(200)	–	Supplier address
WAREHOUSE – Storage locations for stock			
WarehouseID	INT	PK, NOT NULL	Unique warehouse identifier
WarehouseName	VARCHAR(100)	NOT NULL	Name of the warehouse
Location	VARCHAR(100)	–	Town or area of the warehouse
Capacity	INT	CHECK (> 0)	Maximum storage capacity
PRODUCT – Fabrics, threads and raw materials sold			
ProductID	INT	PK, NOT NULL	Unique product identifier
SupplierID	INT	FK → SUPPLIER, NOT NULL	Supplier that provides the product
ProductName	VARCHAR(100)	NOT NULL	Name of the product
ProductType	VARCHAR(50)	CHECK (Fabric, Thread, Raw Material)	Category of the product
UnitPrice	DECIMAL(10,2)	CHECK (>= 0)	Selling price per unit
Color	VARCHAR(30)	–	Color of the product
Size	VARCHAR(30)	–	Roll or batch size (one value)
INVENTORY – Stock of each product in each warehouse			
InventoryID	INT	PK, NOT NULL	Unique inventory record identifier
WarehouseID	INT	FK → WAREHOUSE, NOT NULL	Warehouse holding the stock
ProductID	INT	FK → PRODUCT, NOT NULL	Product being stored
QuantityAvailable	INT	NOT NULL, DEFAULT 0, CHECK (>= 0)	Units currently in stock
LastUpdated	DATE	–	Date the stock was last updated
SALES_ORDER – Orders placed by customers			
OrderID	INT	PK, NOT NULL	Unique order identifier
CustomerID	INT	FK → CUSTOMER, NOT NULL	Customer who placed the order
OrderDate	DATE	–	Date the order was placed
OrderStatus	VARCHAR(20)	CHECK (Pending, Processing, Completed)	Current status of the order
TotalAmount	DECIMAL(12,2)	Derived, stored	Total value of the order
INVOICE – Bills issued to customers			
InvoiceID	INT	PK, NOT NULL	Unique invoice identifier
CustomerID	INT	FK → CUSTOMER, NOT NULL	Customer being billed
InvoiceDate	DATE	–	Date the invoice was issued
TotalAmount	DECIMAL(12,2)	Derived, stored	Amount billed

Status	VARCHAR(20)	CHECK (Paid, Unpaid)	Payment status of the invoice
ORDER_DETAIL – One product line of a sales order			
OrderDetailID	INT	PK, NOT NULL	Unique order line identifier
OrderID	INT	FK → SALES_ORDER, NOT NULL	Order the line belongs to
ProductID	INT	FK → PRODUCT, NOT NULL	Product sold on the line
Quantity	INT	NOT NULL, CHECK (> 0)	Units sold
UnitPrice	DECIMAL(10,2)	–	Price per unit at the time of sale
SubTotal	DECIMAL(12,2)	Derived, stored	Quantity × UnitPrice
PAYMENT – Payments made against an order and invoice			
PaymentID	INT	PK, NOT NULL	Unique payment identifier
OrderID	INT	FK → SALES_ORDER, NOT NULL	Order being paid
InvoiceID	INT	FK → INVOICE, NOT NULL	Invoice being settled
PaymentDate	DATE	NULL while pending	Date the payment was made
Amount	DECIMAL(12,2)	CHECK (>= 0)	Amount paid
PaymentMethod	VARCHAR(30)	CHECK (Bank Transfer, Credit Card, Cash); NULL while pending	How the payment was made
PaymentStatus	VARCHAR(20)	CHECK (Completed, Pending)	Status of the payment

3.6 Normalization

The structure was checked against the three normal forms.

- **First Normal Form (1NF):** every column contains an atomic value and every table has a primary key with no repeating groups. Order lines are stored as rows in ORDER_DETAIL of repeating columns in SALES_ORDER.
- **Second Normal Form (2NF):** every table uses a column primary key so no non-key attribute can depend on part of a key.
- **Third Normal Form (3NF):** key attributes depend only on the key. For example supplier details are stored once in SUPPLIER. Referenced from PRODUCT instead of being repeated on every product row.

Table 3.6: Normalization verification

Table	Highest NF	Justification
CUSTOMER, SUPPLIER, WAREHOUSE	3NF	Each describes one entity; all attributes depend only on the primary key.
PRODUCT	3NF	Supplier data is referenced by SupplierID, not repeated.
INVENTORY	3NF	Resolves the M:N link between warehouse and product; quantity depends on the inventory record.
SALES_ORDER, INVOICE	3NF	Customer details are referenced by CustomerID, not repeated.
ORDER_DETAIL	3NF	Resolves the M:N link between order and product. UnitPrice is kept deliberately to preserve the historical selling price.
PAYMENT	3NF	Each payment depends only on PaymentID and references its order and invoice by key.

Derived values. Two stored totals, SubTotal and TotalAmount, can be calculated from other columns. They are kept intentionally (deliberate denormalization) to speed up reporting and to preserve the figure that was billed, and they are marked as derived in the ER diagram and the data dictionary.

3.7 Physical Database Design (DDL)

The physical design uses Data Definition Language (DDL) to put the schema into Microsoft SQL Server. Tables are made starting with the parent ones so that every foreign key points, to a table that's already there. The foreign key constraints and CHECK constraints are given names that start with FK_ and CK_ so that when there are errors the messages are easy to understand.

```
1  -- PROJECT NAME: B2B Textile Inventory and Sales Management System
2  -- ORGANIZATION: Colour Selection
3
4  -- Create and use the specific database for the organization
5  CREATE DATABASE ColourSelectionDB;
6  GO
7  USE ColourSelectionDB;
8  GO
9
10
11 -- PART 1: DATA DEFINITION LANGUAGE (DDL) - TABLE CREATION & CONSTRAINTS
12
13
14 -- Creating the CUSTOMER table to store client contact information
15 CREATE TABLE CUSTOMER(
16     CustomerID INT PRIMARY KEY, -- Unique identifier for each customer
17     CustomerName VARCHAR(100) NOT NULL,
18     Phone VARCHAR(10),
19     Email VARCHAR(100),
20     Address VARCHAR(100)
21 );
22
23 -- Creating the SUPPLIER table to track where raw materials are sourced from
24 CREATE TABLE SUPPLIER(
25     SupplierID INT PRIMARY KEY,
26     SupplierName VARCHAR(100) NOT NULL,
27     PHONE VARCHAR(10),
28     Email VARCHAR(100),
29     Address VARCHAR(100)
30 );
31
32 -- Creating the WAREHOUSE table to manage different storage locations
33 CREATE TABLE WAREHOUSE(
34     WarehouseID INT PRIMARY KEY,
35     WarehouseName VARCHAR(100) NOT NULL,
36     Location VARCHAR(150),
37     Capacity INT -- Maximum storage capacity of the warehouse
38 );
39
40 -- Creating the PRODUCT table, linked to the Supplier who provides it
41 CREATE TABLE PRODUCT(
42     ProductID INT PRIMARY KEY,
43     SupplierID INT,
44     ProductName VARCHAR(100) NOT NULL,
45     ProductType VARCHAR(50),
46     UnitPrice DECIMAL(10, 2),
47     Color VARCHAR(30),
48     Size VARCHAR(20),
49     FOREIGN KEY (SupplierID) REFERENCES SUPPLIER(SupplierID) -- Relates Product to its Supplier
50 );
51
52 -- Creating the INVENTORY table to track how much of each product is in which warehouse
53 CREATE TABLE INVENTORY(
54     InventoryID INT PRIMARY KEY,
55     WarehouseID INT,
56     ProductID INT,
57     QuantityAvailable INT DEFAULT 0,
58     LastUpdated Date,
59     FOREIGN KEY (WarehouseID) REFERENCES WAREHOUSE(WarehouseID),
60     FOREIGN KEY (ProductID) REFERENCES PRODUCT(ProductID)
61 );
```

```

62
63 -- Creating the SALES_ORDER table to track customer purchases
64 CREATE TABLE SALES_ORDER (
65     OrderID INT PRIMARY KEY,
66     CustomerID INT,
67     OrderDate DATE,
68     OrderStatus VARCHAR(30),
69     TotalAmount DECIMAL(12, 2),
70     FOREIGN KEY (CustomerID) REFERENCES CUSTOMER(CustomerID)
71 );
72
73 -- Creating the INVOICE table to manage billing for sales orders
74 CREATE TABLE INVOICE (
75     InvoiceID INT PRIMARY KEY,
76     CustomerID INT,
77     InvoiceDate DATE,
78     TotalAmount DECIMAL(12, 2),
79     Status VARCHAR(30), -- Tracks if the invoice is 'Paid' or 'Unpaid'
80     FOREIGN KEY (CustomerID) REFERENCES CUSTOMER(CustomerID)
81 );
82
83 -- Creating the ORDER_DETAIL table to track individual line items within a sales order
84 CREATE TABLE ORDER_DETAIL (
85     OrderDetailID INT PRIMARY KEY,
86     OrderID INT,
87     ProductID INT,
88     Quantity INT NOT NULL,
89     UnitPrice DECIMAL(10, 2),
90     SubTotal DECIMAL(12, 2), -- Calculated as Quantity * UnitPrice
91     FOREIGN KEY (OrderID) REFERENCES SALES_ORDER(OrderID),
92     FOREIGN KEY (ProductID) REFERENCES PRODUCT(ProductID)
93 );
94
95 -- Creating the PAYMENT table to track financial transactions against invoices
96 CREATE TABLE PAYMENT (
97     PaymentID INT PRIMARY KEY,
98     OrderID INT,
99     InvoiceID INT,
100    PaymentDate DATE,
101    Amount DECIMAL(12, 2) NOT NULL,
102    PaymentMethod VARCHAR(50),
103    PaymentStatus VARCHAR(30),
104    FOREIGN KEY (OrderID) REFERENCES SALES_ORDER(OrderID),
105    FOREIGN KEY (InvoiceID) REFERENCES INVOICE(InvoiceID)
106 );

```

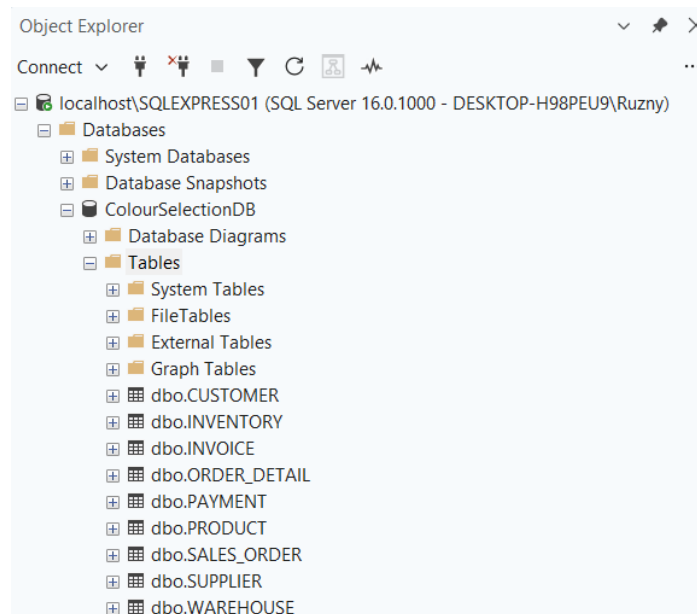


Figure 3.2: ColourSelectionDB and its tables in SQL Server Management Studio

3.8 Data Manipulation language(DML)

Every table has ten records of data. The data includes Sri Lankan customers, local suppliers and international suppliers well as regional warehouses. The data is added in a parent- way so that foreign key references are always correct. Phone numbers are kept without hyphens. The full INSERT statements are, in the SQL script that was submitted. A sample is shown below.

```
112 -- Insert 10 records into CUSTOMER
113 INSERT INTO CUSTOMER (CustomerID, CustomerName, Phone, Email, Address) VALUES
114 (1, 'Ruzny', '0756076855', 'ruznymoulana98@gmail.com', '103/A/5, Sippikulama, Hambantota'),
115 (2, 'Lanka Textiles', '0712223334', 'info@lankatex.lk', '10 Galle Road, Colombo'),
116 (3, 'Kandy Fashions', '0813334445', 'sales@kandyfashions.lk', '45 Dalada Vidiya, Kandy'),
117 (4, 'Metro Apparel', '0114445556', 'contact@metro.lk', '22 Union Place, Colombo 02'),
118 (5, 'Galle Garments', '0915556667', 'purchasing@gallegarments.lk', '18 Fort Road, Galle'),
119 (6, 'Apex Boutique', '0776667778', 'buyer@apex.com', '50 Duplication Rd, Colombo 03'),
120 (7, 'Island Wear', '0727778889', 'orders@islandwear.lk', '12 Beach Rd, Negombo'),
121 (8, 'Crest Fabrics', '0118889990', 'admin@crest.lk', '88 High Level Rd, Nugegoda'),
122 (9, 'Oceanic Styles', '0759990001', 'hello@oceanic.lk', '34 Marine Drive, Colombo 04'),
123 (10, 'Vertex Clothing', '0761112223', 'supply@vertex.com', '90 Kandy Road, Peliyagoda');
```

A verification query confirms the number of records in each table:

```
412 SELECT 'CUSTOMER' AS TableName, COUNT(*) AS Records FROM CUSTOMER UNION ALL
413 SELECT 'SUPPLIER', COUNT(*) FROM SUPPLIER UNION ALL
414 SELECT 'WAREHOUSE', COUNT(*) FROM WAREHOUSE UNION ALL
415 SELECT 'PRODUCT', COUNT(*) FROM PRODUCT UNION ALL
416 SELECT 'INVENTORY', COUNT(*) FROM INVENTORY UNION ALL
417 SELECT 'SALES_ORDER', COUNT(*) FROM SALES_ORDER UNION ALL
418 SELECT 'INVOICE', COUNT(*) FROM INVOICE UNION ALL
419 SELECT 'ORDER_DETAIL', COUNT(*) FROM ORDER_DETAIL UNION ALL
420 SELECT 'PAYMENT', COUNT(*) FROM PAYMENT;
```

Results		Messages
	TableName	Records
1	CUSTOMER	10
2	SUPPLIER	10
3	WAREHOUSE	10
4	PRODUCT	10
5	INVENTORY	10
6	SALES_ORDER	10
7	INVOICE	10
8	ORDER_DETAIL	10
9	PAYMENT	10

Figure 3.3: Record count of every table

3.9 Data Modification (UPDATE)

UPDATE statements modify columns while UPDATE statements keep primary and foreign keys untouched. This approach guarantees that referential integrity is never broken. Two examples are shown below.

```
236 -- Update a Customer's details using their Primary Key
237 UPDATE CUSTOMER
238 SET CustomerName = 'Urban Threads',
239     Phone = '0771234567',
240     Email = 'contact@urbanthreads.com',
241     Address = '789 Apparel Boulevard'
242 WHERE CustomerID = 1;
243
244 -- Update a Supplier's details using their Primary Key
245 UPDATE SUPPLIER
246 SET SupplierName = 'Global Cotton Co.',
247     Phone = '0719876543',
248     Email = 'supply@globalcotton.com',
249     Address = '101 Industrial Estate'
250 WHERE SupplierID = 1;
251
252 -- Update a Warehouse location using its Primary Key
253 UPDATE WAREHOUSE
254 SET WarehouseName = 'East Coast Storage',
255     Location = 'Port City Area',
256     Capacity = 10000
257 WHERE WarehouseID = 1;
```

Deleting data must respect keys: child rows such as PAYMENT, ORDER_DETAIL, INVOICE, SALES_ORDER INVENTORY must be removed before the parent rows they refer to otherwise SQL Server rejects the statement, with error 547 to protect referential integrity. Every DELETE must also have a WHERE clause so that only the intended rows are removed and under the security design (Section 3.11) only the Manager is allowed to delete business records.

3.10 SQL Queries

These queries answer the asked business questions listed in Table 2.5. For each query we show its purpose the SQL statement and a screenshot of its output.

3.10.1 Basic Select Queries

Query B1 – Customer contact list

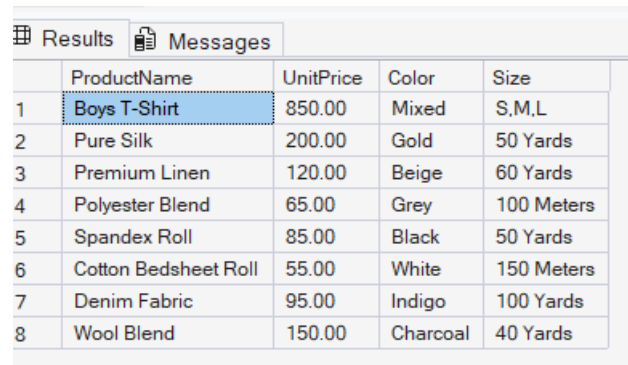
```
279 -- 1. Retrieve all contact information for registered customers
280 SELECT CustomerName, Phone, Email, Address
281 FROM CUSTOMER;
```

Figure 3.4: Output of Query B1

	CustomerName	Phone	Email	Address
1	Ruzny	0756076855	ruznymoulana98@gmail.com	103/A/5, Sippikulama, Hambantota
2	Lanka Textiles	0712223334	info@lankatex.lk	10 Galle Road, Colombo
3	Kandy Fashions	0813334445	sales@kandyfashions.lk	45 Dalada Vidiya, Kandy
4	Metro Apparel	0114445556	contact@metro.lk	22 Union Place, Colombo 02
5	Galle Garments	0915556667	purchasing@gallegarments...	18 Fort Road, Galle
6	Apex Boutique	0776667778	buyer@apex.com	50 Duplication Rd, Colombo 03
7	Island Wear	0727778889	orders@islandwear.lk	12 Beach Rd, Negombo
8	Crest Fabrics	0118889990	admin@crest.lk	88 High Level Rd, Nugegoda
9	Oceanic Styles	0759990001	hello@oceanic.lk	34 Marine Drive, Colombo 04
10	Vertex Clothing	0761112223	supply@vertex.com	90 Kandy Road, Peliyagoda

Query B2 – Products classified as Fabric

```
283 -- 2. Filter products to show only items classified specifically as 'Fabric'
284 SELECT ProductName, UnitPrice, Color, Size
285 FROM PRODUCT
286 WHERE ProductType = 'Fabric';
```

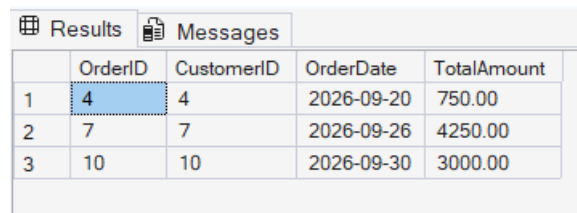


	ProductName	UnitPrice	Color	Size
1	Boys T-Shirt	850.00	Mixed	S,M,L
2	Pure Silk	200.00	Gold	50 Yards
3	Premium Linen	120.00	Beige	60 Yards
4	Polyester Blend	65.00	Grey	100 Meters
5	Spandex Roll	85.00	Black	50 Yards
6	Cotton Bedsheet Roll	55.00	White	150 Meters
7	Denim Fabric	95.00	Indigo	100 Yards
8	Wool Blend	150.00	Charcoal	40 Yards

Figure 3.5: Output of Query B2

Query B3 – Pending sales orders to follow up

```
288 -- 3. Identify sales orders that are currently marked as 'Pending' to follow up
289 SELECT OrderID, CustomerID, OrderDate, TotalAmount
290 FROM SALES_ORDER
291 WHERE OrderStatus = 'Pending';
```

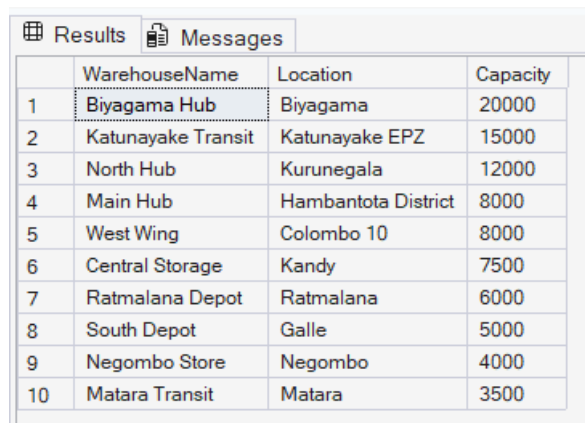


	OrderID	CustomerID	OrderDate	TotalAmount
1	4	4	2026-09-20	750.00
2	7	7	2026-09-26	4250.00
3	10	10	2026-09-30	3000.00

Figure 3.6: Output of Query B3

Query B4 – Storage capacity of each warehouse, largest first

```
293 -- 4. Display the total storage capacity across all active warehouse, Largest first.
294 SELECT WarehouseName, Location, Capacity
295 FROM WAREHOUSE
296 ORDER BY Capacity DESC;
```



	WarehouseName	Location	Capacity
1	Biyagama Hub	Biyagama	20000
2	Katunayake Transit	Katunayake EPZ	15000
3	North Hub	Kurunegala	12000
4	Main Hub	Hambantota District	8000
5	West Wing	Colombo 10	8000
6	Central Storage	Kandy	7500
7	Ratmalana Depot	Ratmalana	6000
8	South Depot	Galle	5000
9	Negombo Store	Negombo	4000
10	Matara Transit	Matara	3500

Figure 3.7: Output of Query B4

Query B5 – Unpaid invoices for the finance team

```
298 -- 5. List all unpaid invoices so the finance team can track pending payments
299 SELECT InvoiceID, CustomerID, InvoiceDate, TotalAmount
300 FROM INVOICE
301 WHERE Status = 'Unpaid';
```

Results		Messages		
	InvoiceID	CustomerID	InvoiceDate	TotalAmount
1	4	4	2026-09-21	750.00
2	6	6	2026-09-25	6500.00
3	7	7	2026-09-26	4250.00
4	9	9	2026-09-30	4750.00
5	10	10	2026-09-30	3000.00

Figure 3.8: Output of Query B5

3.10.2 Advanced SELECT Queries (GROUP BY, HAVING, ORDER BY)

Query A1 – High-value customers (more than 4,000 ordered in total)

```
308 -- 1. Identify high-value customers who have ordered more than 4000 in total value
309 SELECT CustomerID, SUM(TotalAmount) AS TotalOrdered
310 FROM SALES_ORDER
311 GROUP BY CustomerID
312 HAVING SUM(TotalAmount) > 4000
313 ORDER BY TotalOrdered DESC;
```

Results		Messages	
	CustomerID	TotalOrdered	
1	1	15000.00	
2	3	10000.00	
3	6	6500.00	
4	9	4750.00	
5	2	4500.00	
6	7	4250.00	

Figure 3.9: Output of Query A1

Query A2 – Product types with an average unit price above 50

```
315 -- 2. Find product categories where the average unit price exceeds 50
316 SELECT ProductType, AVG(UnitPrice) AS AveragePrice
317 FROM PRODUCT
318 GROUP BY ProductType
319 HAVING AVG(UnitPrice) > 50
320 ORDER BY AveragePrice DESC;
```

Results		Messages	
	ProductType	AveragePrice	
1	Fabric	202.500000	

Figure 3.10: Output of Query A2

Query A3 – Warehouses currently storing more than 400 items

```
322 -- 3. Identify large-capacity warehouses currently storing more than 400 items
323 SELECT WarehouseID, SUM(QuantityAvailable) AS TotalItemsStored
324 FROM INVENTORY
325 GROUP BY WarehouseID
326 HAVING SUM(QuantityAvailable) > 400
327 ORDER BY TotalItemsStored DESC;
```

	WarehouseID	TotalItemsStored
1	4	1000
2	6	800
3	8	600
4	2	500
5	7	450

Figure 3.11: Output of Query A3

Query A4 – Top-selling products (more than 40 units sold)

```
329 -- 4. Calculate total quantity sold per product, only showing top-selling items (over 40 units)
330 SELECT ProductID, SUM(Quantity) AS TotalUnitsSold
331 FROM ORDER_DETAIL
332 GROUP BY ProductID
333 HAVING SUM(Quantity) > 40
334 ORDER BY TotalUnitsSold DESC;
```

	ProductID	TotalUnitsSold
1	6	100
2	2	100
3	3	50
4	4	50
5	1	50
6	7	50
7	8	50
8	9	50

Figure 3.12: Output of Query A4

Query A5 – Number of invoices per status (at least three each)

```
336 -- 5. Count how many invoices exist per status, filtering for statuses with at least 3 occurrences
337 SELECT Status, COUNT(InvoiceID) AS NumberOfInvoices
338 FROM INVOICE
339 GROUP BY Status
340 HAVING COUNT(InvoiceID) >= 3
341 ORDER BY NumberOfInvoices DESC;
```

	Status	NumberOfInvoices
1	Paid	5
2	Unpaid	5

Figure 3.13: Output of Query A5

3.10.3 JOIN Queries

Query J1 – Inventory location report (INVENTORY, WAREHOUSE and PRODUCT)

```
348  -- 1. Inventory Location Report: Joins INVENTORY, WAREHOUSE, and PRODUCT
349  -- To display which specific products are stored in which warehouse locations
350  SELECT w.WarehouseName, p.ProductName, p.Color, i.QuantityAvailable
351  FROM INVENTORY i
352  JOIN WAREHOUSE w ON i.WarehouseID = w.WarehouseID
353  JOIN PRODUCT p ON i.ProductID = p.ProductID;
```

	WarehouseName	ProductName	Color	QuantityAvailable
1	Main Hub	Boys T-Shirt	Mixed	100
2	West Wing	Raw Cotton Roll	White	500
3	South Depot	Pure Silk	Gold	150
4	North Hub	Nylon Thread Batch	Black	1000
5	Central Storage	Premium Linen	Beige	300
6	Katunayake Transit	Polyester Blend	Grey	800
7	Negombo Store	Spandex Roll	Black	450
8	Biyagama Hub	Cotton Bedsheet Roll	White	600
9	Ratmalana Depot	Denim Fabric	Indigo	250
10	Matara Transit	Wool Blend	Charcoal	120

Figure 3.14: Output of Query J1

Query J2 – Customer orders with item details (CUSTOMER, SALES_ORDER)

```
355  -- 2. Customer Order Report: Joins CUSTOMER and SALES_ORDER
356  -- To show the names of customers alongside the specific dates and statuses of their orders
357  SELECT c.CustomerName, s.OrderID, s.OrderDate, s.OrderStatus, s.TotalAmount
358  FROM CUSTOMER c
359  JOIN SALES_ORDER s ON c.CustomerID = s.CustomerID;
```

	CustomerName	OrderID	OrderDate	OrderStatus	TotalAmount
1	Ruzny	1	2026-09-10	Completed	15000.00
2	Lanka Textiles	2	2026-09-15	Completed	4500.00
3	Kandy Fashions	3	2026-09-18	Completed	10000.00
4	Metro Apparel	4	2026-09-20	Pending	750.00
5	Galle Garments	5	2026-09-22	Completed	3600.00
6	Apex Boutique	6	2026-09-25	Processing	6500.00
7	Island Wear	7	2026-09-26	Pending	4250.00
8	Crest Fabrics	8	2026-09-28	Completed	2750.00
9	Oceanic Styles	9	2026-09-29	Processing	4750.00
10	Vertex Clothing	10	2026-09-30	Pending	3000.00

Figure 3.15: Output of Query J2

Query J3 – Detailed Sales Itemization (ORDER_DETAIL and PRODUCT)

```

361 -- 3. Detailed Sales Itemization: Joins ORDER_DETAIL and PRODUCT
362 -- To show a breakdown of individual items sold, including product names and subtotal calculations
363 SELECT od.OrderID, p.ProductName, p.UnitPrice, od.Quantity, od.SubTotal
364 FROM ORDER_DETAIL od
365 JOIN PRODUCT p ON od.ProductID = p.ProductID;

```

	OrderID	ProductName	UnitPrice	Quantity	SubTotal
1	1	Boys T-Shirt	850.00	50	42500.00
2	2	Raw Cotton Roll	45.00	100	4500.00
3	3	Pure Silk	200.00	50	10000.00
4	4	Nylon Thread Batch	15.00	50	750.00
5	5	Premium Linen	120.00	30	3600.00
6	6	Polyester Blend	65.00	100	6500.00
7	7	Spandex Roll	85.00	50	4250.00
8	8	Cotton Bedsheet Roll	55.00	50	2750.00
9	9	Denim Fabric	95.00	50	4750.00
10	10	Wool Blend	150.00	20	3000.00

Figure 3.16: Output of Query J3

3.10.4 Database Security (DCL)

Two logins and database users were created. The Manager gets data rights. The Clerk gets read access to master data. The Clerk also gets insert and update rights for transactions.. There is an explicit DENY, on deleting business records. In SQL Server, a DENY always overrides a GRANT. So the Clerk can never delete these records.

Table 3.7: Database user privileges

Table	ManagerUser	ClerkUser	Clerk restriction
PRODUCT, WAREHOUSE, SUPPLIER	SELECT, INSERT, UPDATE, DELETE	SELECT	Master data is read-only
CUSTOMER, SALES_ORDER, INVOICE, PAYMENT	SELECT, INSERT, UPDATE, DELETE	SELECT, INSERT, UPDATE	DENY DELETE
ORDER_DETAIL	SELECT, INSERT, UPDATE, DELETE	SELECT, INSERT, UPDATE	No delete rights granted
INVENTORY	SELECT, INSERT, UPDATE, DELETE	SELECT, UPDATE	Cannot add or delete stock records

```

372 -- 1. Create Logins for Server-level access
373 CREATE LOGIN TextileManager WITH PASSWORD = 'Manager123!';
374 CREATE LOGIN TextileClerk WITH PASSWORD = 'Clerk123!';
375
376 -- 2. Map Logins to Database Users within ColourSelectionDB
377 CREATE USER ManagerUser FOR LOGIN TextileManager;
378 CREATE USER ClerkUser FOR LOGIN TextileClerk;
379
380 -- 3. Grant Administrative Permissions for Manager (High-Level Access)
381 -- Managers receive full CRUD (Create, Read, Update, Delete) rights across all tables.
382 GRANT SELECT, INSERT, UPDATE, DELETE ON CUSTOMER TO ManagerUser;
383 GRANT SELECT, INSERT, UPDATE, DELETE ON SUPPLIER TO ManagerUser;
384 GRANT SELECT, INSERT, UPDATE, DELETE ON PRODUCT TO ManagerUser;
385 GRANT SELECT, INSERT, UPDATE, DELETE ON WAREHOUSE TO ManagerUser;
386 GRANT SELECT, INSERT, UPDATE, DELETE ON INVENTORY TO ManagerUser;
387 GRANT SELECT, INSERT, UPDATE, DELETE ON SALES_ORDER TO ManagerUser;
388 GRANT SELECT, INSERT, UPDATE, DELETE ON INVOICE TO ManagerUser;
389 GRANT SELECT, INSERT, UPDATE, DELETE ON ORDER_DETAIL TO ManagerUser;
390 GRANT SELECT, INSERT, UPDATE, DELETE ON PAYMENT TO ManagerUser;
391
392 -- 4. Grant Restricted Operational Permissions for Clerk
393 -- Clerks can view general static data but cannot modify foundational tables (Products, Warehouses, Suppliers).
394 GRANT SELECT ON PRODUCT TO ClerkUser;
395 GRANT SELECT ON WAREHOUSE TO ClerkUser;
396 GRANT SELECT ON SUPPLIER TO ClerkUser;
397
398 -- Clerks process daily transactions, so they are granted Insert and Update rights on operational tables.
399 GRANT SELECT, INSERT, UPDATE ON CUSTOMER TO ClerkUser;
400 GRANT SELECT, INSERT, UPDATE ON SALES_ORDER TO ClerkUser;
401 GRANT SELECT, INSERT, UPDATE ON ORDER_DETAIL TO ClerkUser;
402 GRANT SELECT, INSERT, UPDATE ON INVOICE TO ClerkUser;
403 GRANT SELECT, INSERT, UPDATE ON PAYMENT TO ClerkUser;
404 GRANT SELECT, UPDATE ON INVENTORY TO ClerkUser;
405
406 -- 5. Explicitly DENY Permissions (Security Best Practice)
407 -- Ensure Clerks cannot accidentally or maliciously delete financial/business records.
408 DENY DELETE ON CUSTOMER TO ClerkUser;
409 DENY DELETE ON SALES_ORDER TO ClerkUser;
410 DENY DELETE ON INVOICE TO ClerkUser;
411 DENY DELETE ON PAYMENT TO ClerkUser;

```

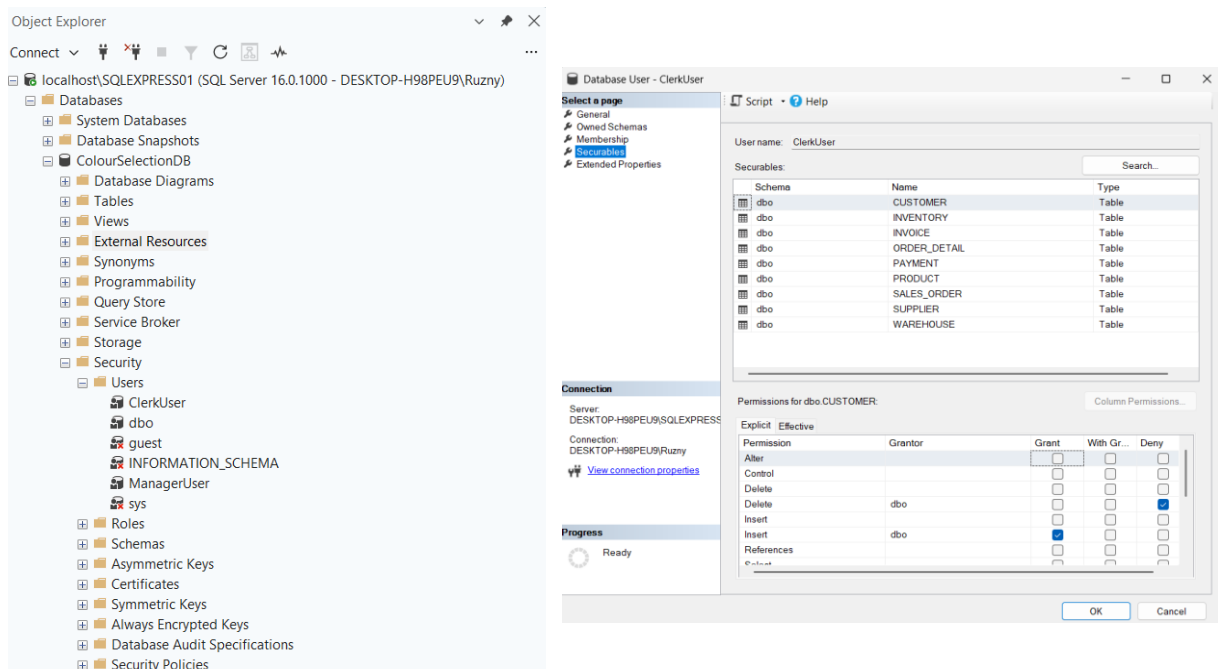


Figure 3.17: Database users and permissions in SSMS

Chapter 4: Conclusion

4.1 Summary

The project delivered a database, for Colour Selection, a B2B textile trading business. The project design moved from an ER model of nine entities through a normalized relational schema to a working SQL Server database that holds realistic data is queried by thirteen business statements and is protected by role-based security. The database gives management a consistent view of suppliers, products, warehouse stock, customer orders, invoices and payments which replaces the scattered spreadsheets and ledgers described in the problem statement.

4.2 Achievement of Objectives

Objective	How it was achieved
Model at least eight entities with an ER diagram	Nine entities with 1:N cardinality ratios and participation constraints (Sections 3.1 to 3.3).
Convert the ER model into a normalized schema	Relational schema, data dictionary and 3NF verification (Sections 3.4 to 3.6).
Create the physical database	ColourSelectionDB with nine tables and key and CHECK constraints (Section 3.7).
Populate at least ten records per table	Ten records inserted into every table and verified (Section 3.8).
Answer FAQs with SQL queries	Five basic, five advanced and three JOIN queries (Section 3.10).
Secure the data	Manager and Clerk users with GRANT and DENY permissions (Section 3.11).

4.3 Final remarks

The project helped us learn about the full database lifecycle starting from looking at a business to putting the database in place and making sure it was safe. Setting a plan making the design follow normal rules and testing each table with real questions made a strong base. This base allows Colour Selection to handle its inventory and sales correctly and expand its work with confidence. The design has room to grow later by adding a front-end app connecting each invoice to its sales order and adding stored procedures and triggers to keep the totals updated automatically.

Contribution Table

Name	Index No	Specific Contributions
S.R.R MOULANA	CODSE261F-036	Advanced SQL queries, Role-based security setup, JOIN queries
A.M.A ZIMTHI	CODSE261F-033	ER Diagram design, DDL script writing, Report formatting
S.W EKANAYAKE	CODSE261F-040	Normalization (1NF to 3NF), Data Dictionary creation
K.P.H DILRUWAN	CODSE261F-055	Data insertion (DML), Presentation